◦ The resulting attestation_elements_message, including optional fields, SHALL be no more than RESP_MAX bytes once serialized.

```
attestation_elements_message =
{
    certification_declaration(1) = certification_declaration,
    attestation_nonce(2)         = attestation_nonce,
    timestamp(3)                 = timestamp,
    firmware_information(4)      = firmware_information,

    ... optional fields per attestation-elements ..
}
```

2. Obtain the AttestationChallenge from a CASE session, resumed CASE session, or PASE session depending on the method used to establish the current session. This is an octet string of length CRYPTO_SYMMETRIC_KEY_LENGTH_BITS. Save it for the next step as attestation_challenge.
3. Locally generate an attestation_tbs message as an octet string by concatenating the attestation_elements_message and the attestation_challenge:
   ◦ attestation_tbs = attestation_elements_message || attestation_challenge
4. Compute the attestation_signature and record it as an ec-signature octet string:

```
attestation_signature = Crypto_Sign(
    message = attestation_tbs,
    privateKey = Device Attestation Private Key
)
```

5. Fill the AttestationElements field of the AttestationResponse Command using the contents of the attestation_elements_message octet string.
6. Fill the AttestationSignature field of the AttestationResponse Command using the contents of the attestation_signature octet string.
7. Note that the attestation_challenge SHALL NOT be included in any of the payloads conveyed as part of the Attestation Information.

See Section F.2, “Device Attestation Response test vector” for an example computation of the above messages and application payloads.

11.18.4.8. NOCSR Elements

The NOCSR Elements contain the metadata related to NOCSR, encoded in Matter TLV.

NOCSR Elements TLV

```
nocsr-elements => STRUCTURE [ tag-order ]
{
```

